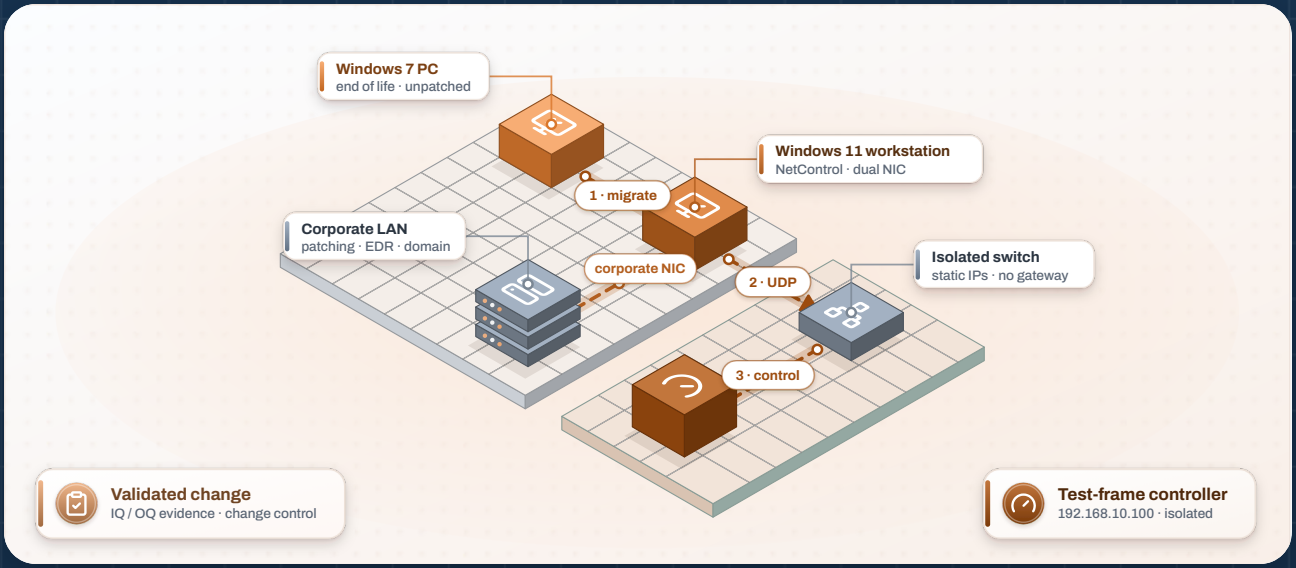
STANDARD OPERATING PROCEDURE · FIELD GUIDE SERIES

SOP-LAB-001

Modernizing a Windows 7 Lab Application onto Windows 11

Move an instrument-control application off an end-of-life OS in a regulated lab, four ways (direct install with compatibility mode, compatibility shims, an isolated Hyper-V Windows 7 VM, and the vendor upgrade path), with a validated instrument network and IQ/OQ evidence.

Lab Systems · Windows 11 · App Compatibility · GxP Change Control



DOCUMENT ID	SOP-LAB-001
VERSION	2.0
EFFECTIVE	October 2026
AUDIENCE	Lab IT and endpoint engineers, validation / CSV specialists, lab system owners
AUTHOR	Richmond Dieujuste, Principal Consultant

Rich Dieu IT Solutions LLC · richdieu.com

Sanitized reference edition — client identifiers removed

Contents

1. Concepts You Need First 3

1.1 Why an end-of-life OS is a problem in a regulated lab 3

1.2 How Windows keeps old applications working 4

1.3 Instrument networking: isolated NIC, static IP, UDP 6

1.4 The virtual-machine fallback and its own end of life 7

1.5 Validation and change control in one page 7

1.6 Choosing a method 8

2. Access, Prerequisites and Planning 10

2.1 Level of access required 10

2.2 Prerequisites checklist 10

2.3 Risks and how this SOP controls them 11

3. Common Foundation · Workstation and Instrument Network 12

3.1 Base template · prepare the workstation with PowerShell 12

4. Method A · Direct Install with Compatibility Mode 15

5. Method B · Compatibility Shims at Scale 17

5.1 Option 1 · Compatibility layers in the registry 17

5.2 Option 2 · A custom shim database with Compatibility Administrator 17

6. Methods C and D · Hyper-V Fallback and Vendor Upgrade 19

6.1 Method C · An isolated Windows 7 VM in Hyper-V 19

6.2 Method D · The vendor upgrade path 20

7. Verification, Rollback and Troubleshooting 22

7.1 Operational checks (OQ) 22

7.2 Evidence checklist (IQ and OQ) 23

7.3 Rollback 23

7.4 Troubleshooting 24

8. Summary, Glossary and Review 26

8.1 Chapter summary 26

8.2 Glossary 26

LEARNING OBJECTIVES

After working through this procedure you will be able to:

- Explain why an end-of-life operating system is a compliance and security risk in a GxP lab, and why lab PCs fall behind.
- Describe how Windows application compatibility works: compatibility modes, shims, the Program Compatibility Assistant and custom shim databases.
- Configure an isolated instrument network interface with a static address, no gateway, the right firewall profile and UDP rules.
- Deliver the application by direct install, compatibility shims, an isolated Hyper-V Windows 7 VM, or a vendor upgrade, and choose between them.
- Produce installation and operational qualification (IQ/OQ) evidence and run the change through change control.
- Diagnose dropouts and failures, and roll back without losing the validated state.

TIME	DIFFICULTY	MINIMUM ACCESS	RISK
1–2 days incl. testing (excl. approvals)	Intermediate–advanced	Local admin on the lab PC + change owner	Medium · validated instrument

CHAPTER 1

Concepts You Need First

Laboratories run on instruments, and instruments run on software that is often older than the people operating them. In the reference engagement, a medical-device R&D lab controlled its orthopaedic tribology test frames (knee and hip joint simulators) from workstations running Windows 7, because the control software, AMTI NetControl, had only ever been qualified on Windows 7. Earlier attempts on Windows 10 had failed: the software launched, but the connection to the test frame kept dropping. The task was to prove that the application could run reliably on Windows 11, without breaking the hardware link or the lab's validated state, and to leave a repeatable path for every similar workstation in the lab.

1.1 Why an end-of-life OS is a problem in a regulated lab

Every Windows release has a published lifecycle. After **end of support** Microsoft releases no further security updates (except through paid Extended Security Updates, which for Windows 7 ended in January 2023). An unpatched PC on a corporate network is an open door: worms such as WannaCry spread precisely through unpatched Windows 7 machines.

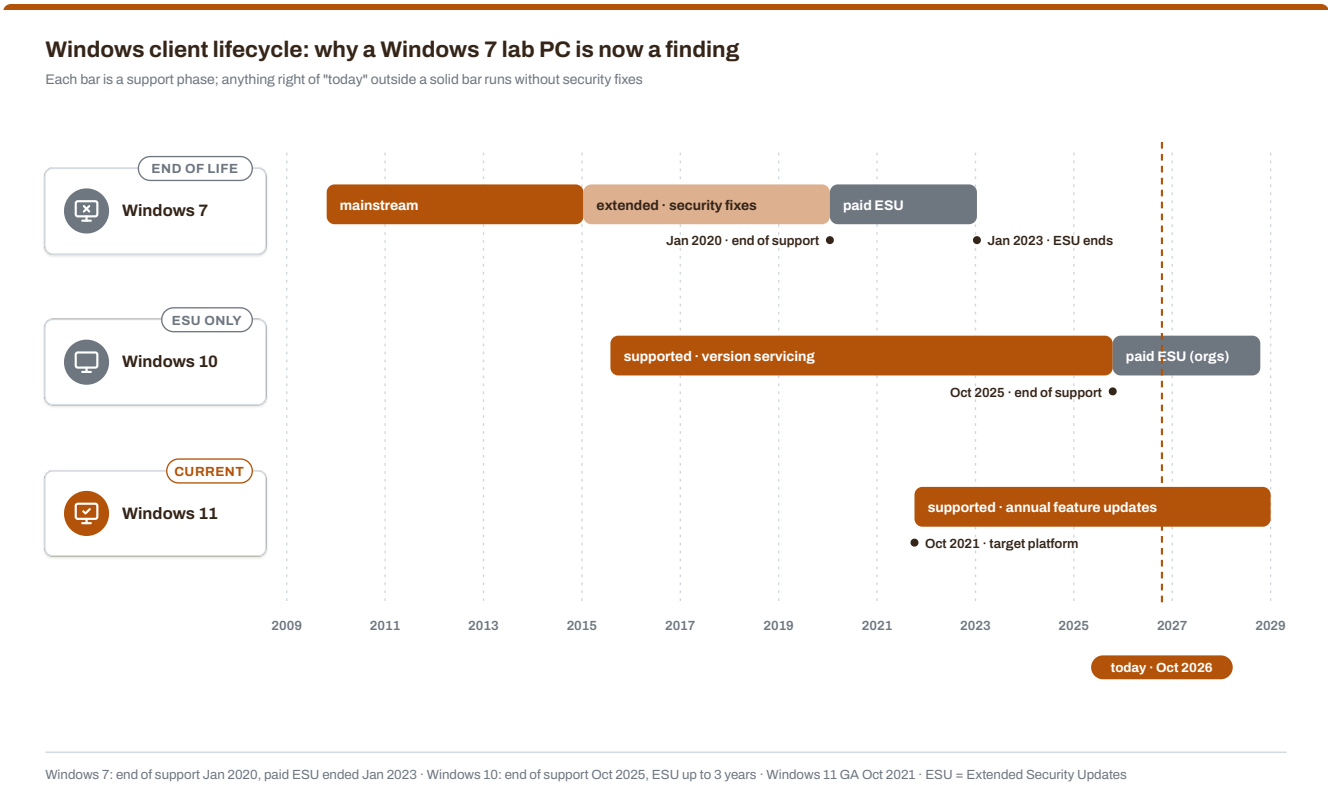


Figure 1. Windows 7 has received no security fixes of any kind since January 2023, so a lab PC still running it is an open finding rather than a legacy curiosity.

In a **GxP** environment (Good Laboratory, Clinical and Manufacturing Practice), the problem is also regulatory. Computerised systems that create or hold quality data must be validated and kept in a state of control: 21 CFR Part 11 in the US, EU GMP Annex 11 in Europe, and industry guidance such as GAMP 5. An unsupported OS undermines that control, and auditors increasingly raise it as a finding. Yet lab PCs fall behind for understandable reasons:

- The instrument vendor only certifies specific OS versions, and the controller's drivers may not exist for newer Windows.
- Every change to a validated system needs change control, risk assessment and re-qualification, which costs time and money.
- Instruments are expensive and long-lived; the PC is an accessory that nobody budgets to replace.

WHY IT MATTERS · MODERNIZE DELIBERATELY, NOT BY ACCIDENT

The safest path is not "never touch it" and not "upgrade and hope". It is a planned, documented change: prove the application on the new OS in a controlled test, capture objective evidence, and release it through change control. This SOP is that plan.

1.2 How Windows keeps old applications working

Most Windows 7 applications run unchanged on Windows 11, because the Win32 API is remarkably stable. When something does break, the usual causes are: installers that check the OS version and refuse to continue, programs that write to protected folders (`C:\Program Files`) or `HKLM` as a standard user, hard-coded paths, old drivers, and assumptions about the network stack. Windows has a built-in mechanism for these cases: the **application compatibility shim engine**.

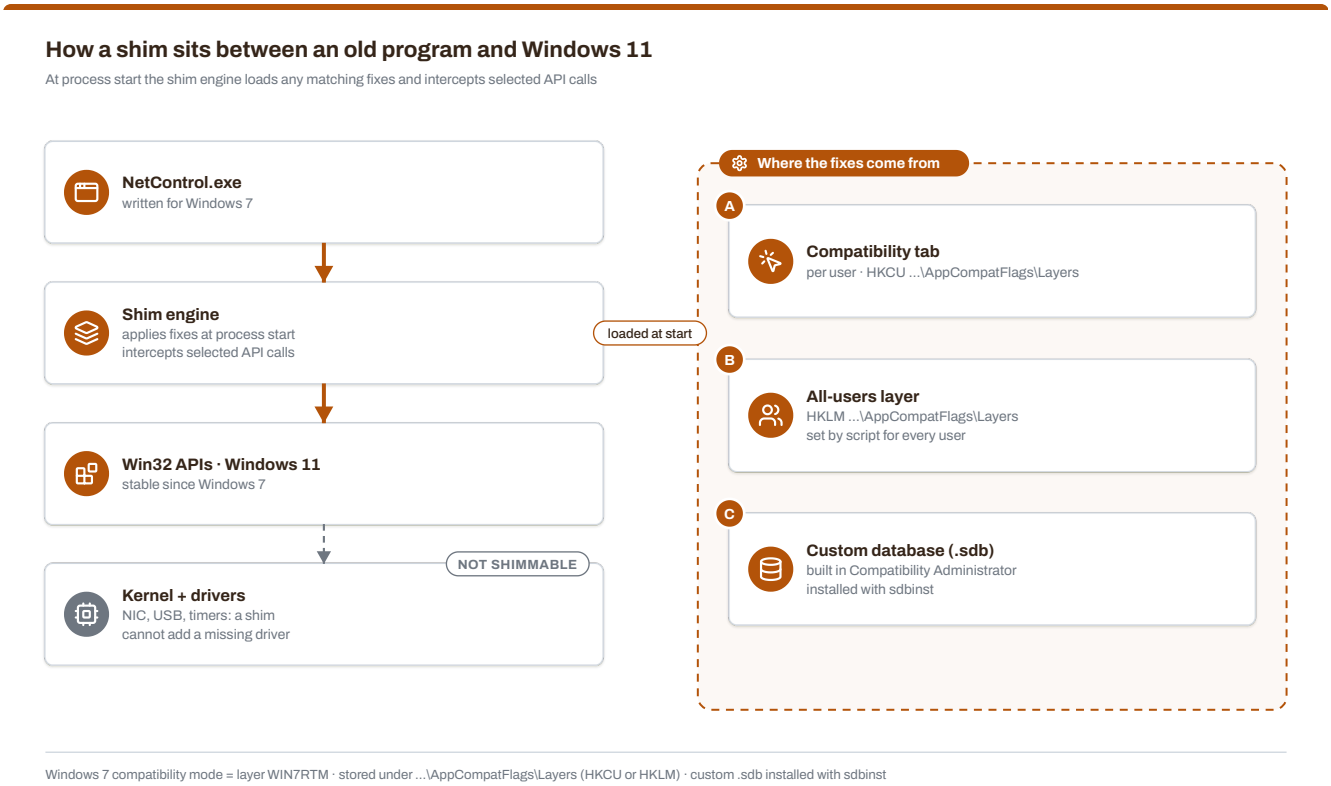


Figure 2. A shim can change how Windows answers an old program, but it cannot supply a driver the hardware no longer has.

KEY CONCEPT · A SHIM IS A POLITE LIE

A *shim* is a small piece of code that sits between an application and Windows and adjusts the conversation. "Windows 7 compatibility mode" is simply a bundle of shims (the layer `WIN7RTM`) that, among other things, tells the program it is running on Windows 7. It changes how Windows answers the application; it does not install an old Windows, and it cannot fix a missing kernel driver.

Mechanism	What it does
Compatibility tab	Properties › Compatibility on an .exe: compatibility mode, run as administrator, DPI overrides. Stored per user (or for all users) under `AppCompatFlags\Layers`.
Program Compatibility Assistant (PCA)	A Windows service (`PcaSvc`) that notices known failure patterns, such as an installer that failed because of the OS version, and offers to re-run it with compatibility settings.
Custom shim database (.sdb)	Built with Compatibility Administrator (Windows ADK). Applies specific fixes to a specific executable on every machine where it is installed with `sdbinst`; the enterprise way to deploy shims.

1.3 Instrument networking: isolated NIC, static IP, UDP

Many lab instruments are controlled over Ethernet rather than USB. The PC and the instrument's controller form a tiny private network, often just a cable or a small unmanaged switch. Best practice is a **dedicated second network interface (NIC)** for the instrument, separate from the NIC that connects to the corporate network.

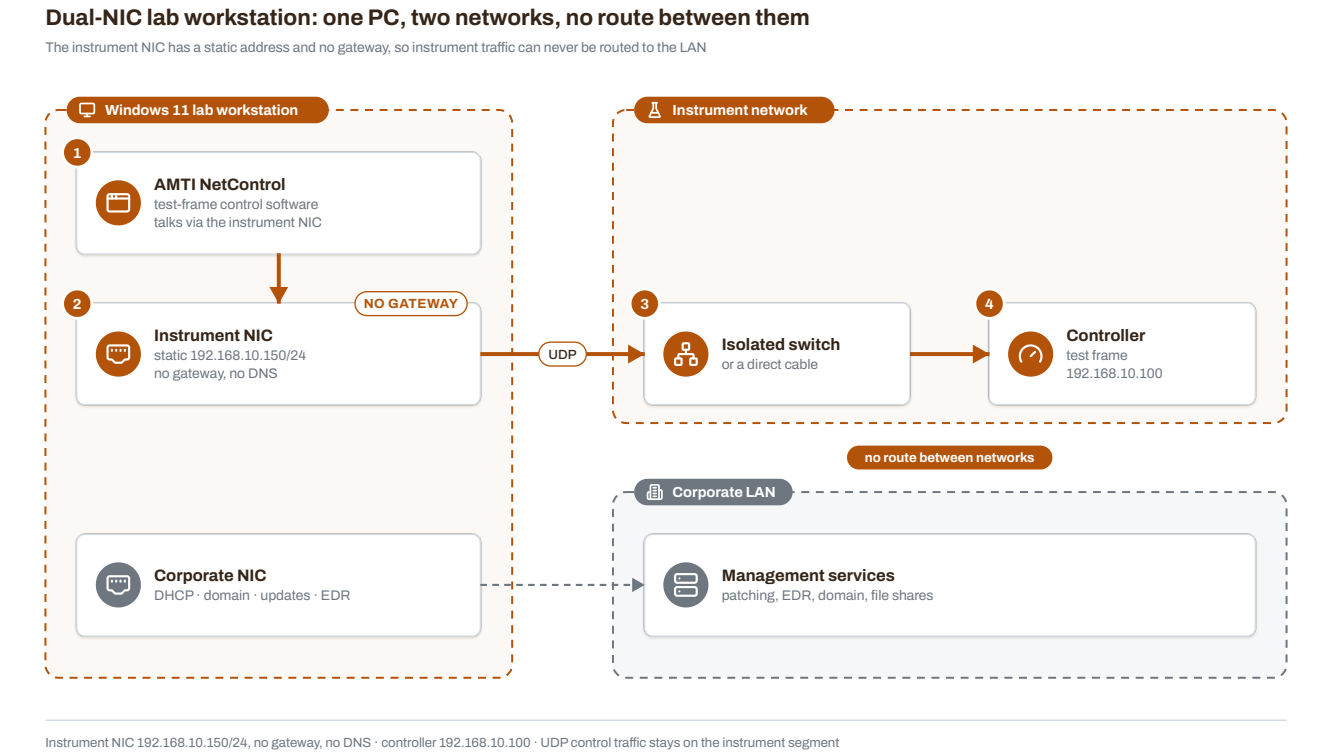


Figure 3. Keeping the instrument NIC without a gateway is what makes the dual-homed PC safe: nothing on the instrument side can be routed to the corporate LAN.

- **Static address, no gateway**: there is no DHCP server on a cable, and a default gateway on the second NIC would make Windows try to route corporate traffic into the instrument network.
- **UDP**: real-time control software often uses the User Datagram Protocol, which sends packets without a connection or retransmission. It is fast and predictable, but nothing tells the application that packets were dropped; they are simply missing. Firewalls, power saving and link problems therefore show up as "random disconnects".
- **Network profile**: Windows labels each network *Domain*, *Private* or *Public*. An instrument network has no domain controller, so Windows calls it an *Unidentified network* and applies the restrictive **Public** firewall profile. Inbound UDP from the controller is then blocked unless a rule allows it.

WATCH OUT · "IT RUNS BUT KEEPS DISCONNECTING"

That was exactly the symptom in the earlier Windows 10 attempts. With UDP, the most common causes are the Public firewall profile blocking inbound replies, NIC power management putting the adapter to sleep, and speed/duplex negotiation problems with old 10 Mbps controllers. Chapter 3 configures all three deliberately.

1.4 The virtual-machine fallback and its own end of life

If the application cannot be made to work on Windows 11, a common fallback is to run Windows 7 *inside* a Hyper-V virtual machine on a Windows 11 host, with the instrument NIC handed to the VM through a virtual switch. The host is modern and patched; the legacy software sees the OS it expects. But the VM is still Windows 7: it gets no security updates, so it must be isolated from every network except the instrument's, and it is a bridge to a vendor upgrade, not a destination.

1.5 Validation and change control in one page

A validated system changes only through **change control**: a documented request, a risk assessment and approval, testing against pre-approved acceptance criteria, and a summary report signed by Quality. Testing is commonly structured as:

- **IQ, installation qualification**: the system is installed as specified (OS build, application version, drivers, network settings).
- **OQ, operational qualification**: it functions as intended across its operating range (the application connects, controls and acquires data, with no dropouts over a defined run).
- **PQ, performance qualification**: it performs in real use (a real test on a real fixture).

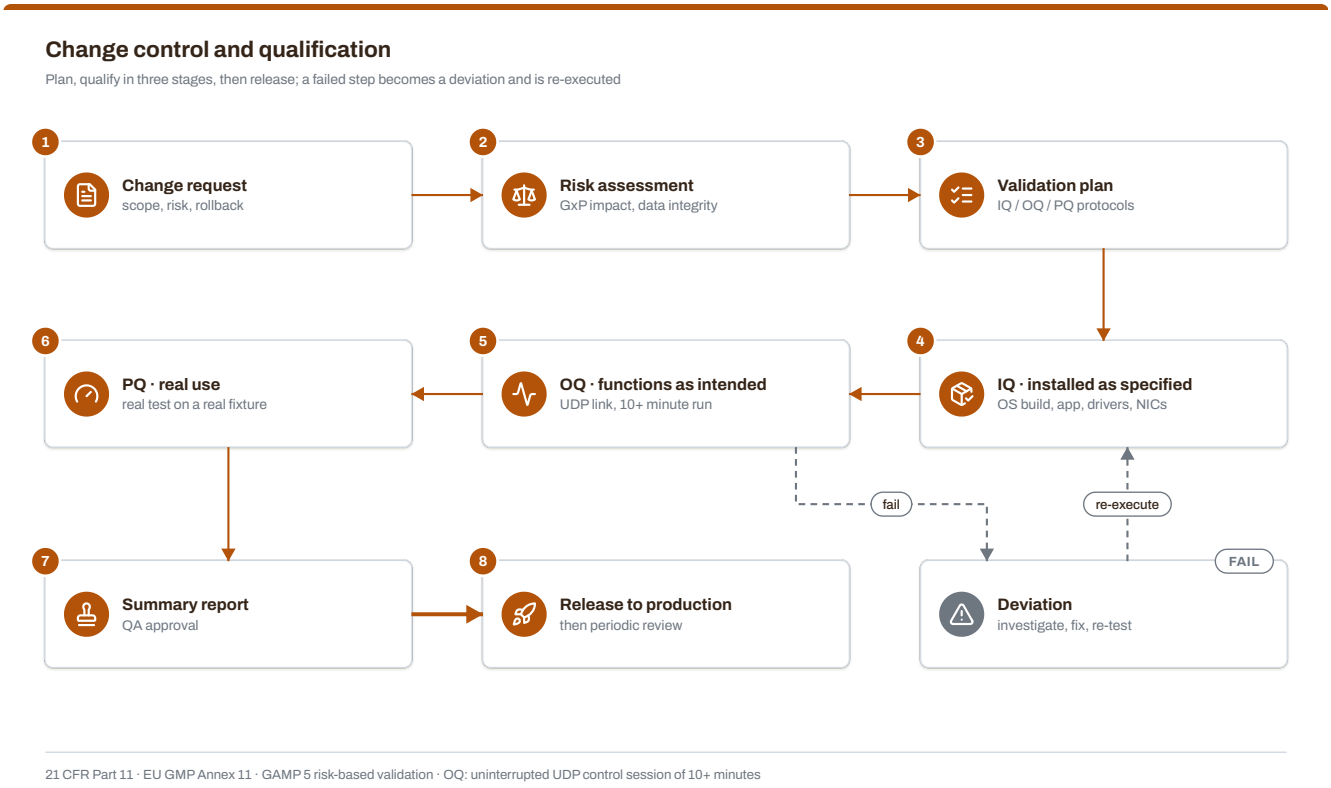


Figure 4. In a regulated lab the migration is a controlled change: qualification evidence, not a working demo, is what allows release.

Current FDA thinking on Computer Software Assurance (CSA) encourages risk-based, less script-heavy testing focused on what could actually affect product quality or data integrity. The lab's quality system decides the exact format; this SOP produces the evidence any of them need.

1.6 Choosing a method

A · Direct install

Install on Windows 11, using compatibility mode for the installer or program if needed.

BEST FOR: APPS THAT JUST NEED A NUDGE

B · Compatibility shims

Set layers for all users by script, or build a shim database with Compatibility Administrator.

BEST FOR: REPEATABLE, FLEET-WIDE FIXES

C · Hyper-V Windows 7 VM

Run the legacy stack in an isolated VM with the instrument NIC on an external vSwitch.

BEST FOR: APPS THAT CANNOT RUN ON WIN 11

D · Vendor upgrade

Move to a vendor-supported version for Windows 11, with any firmware changes.

BEST FOR: THE LONG-TERM ANSWER

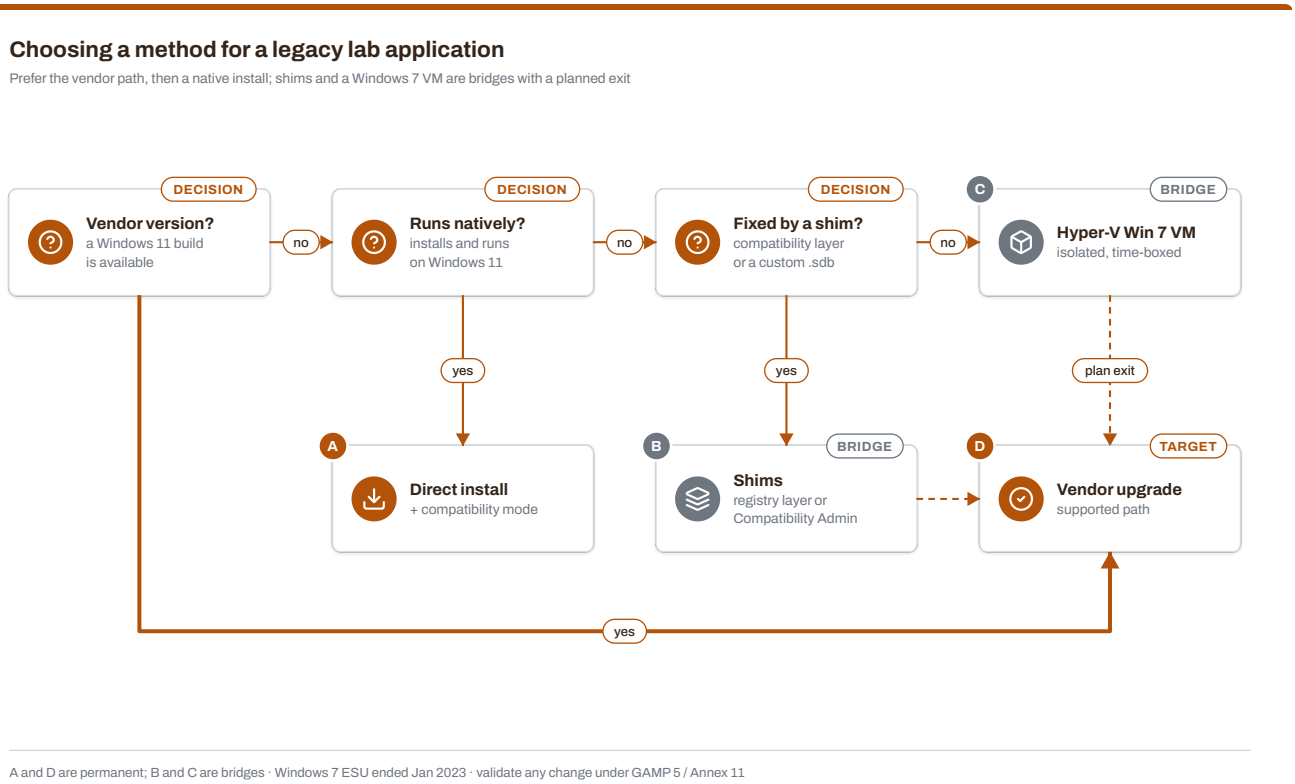


Figure 5. Shims and a Windows 7 VM keep the instrument running, but only a vendor-supported version ends the risk, so plan that exit from day one.

CHAPTER 2

Access, Prerequisites and Planning

2.1 Level of access required

Task	Least-privilege access	Notes
Raise, approve and own the change	Change owner / requester role in the ITSM tool; Quality approver for GxP changes	In the reference engagement, a lack of change-owner rights was the first blocker; assign it early.
Remove from domain, decommission and reimage	Endpoint / imaging team; delegated rights on the computer object's OU	Requested through the ITSM / ServiceNow request portal.
Install application, set NIC, firewall rules, HKLM compatibility flags	Local **Administrators** on the lab workstation	Use a LAPS-managed local admin or a time-bound admin group membership.
Enable Hyper-V	Local Administrators (feature install + restart)	Requires Windows 11 Pro, Enterprise or Education and hardware virtualization enabled in firmware.
Manage the VM day to day	**Hyper-V Administrators** (local group)	Lets lab IT start, stop and checkpoint VMs without full admin.
Vendor remote support	Session through the privileged remote access gateway (e.g. BeyondTrust PRA), recorded	Arrange the remote-access group's approval in advance; it delayed testing in the reference case.
Instrument switch / cabling	Lab facilities or local network team	The instrument network is not routed; no firewall change should be needed.

GOOD PRACTICE · SPLIT THE CHANGE

Break the work into small, separately approved changes: compatibility-mode testing on one machine, VM testing on another, and the production rollout last. Small changes get approved faster and a failure in one does not invalidate the others.

2.2 Prerequisites checklist

- ☐ Approved change request and validation plan with acceptance criteria (for example "no dropouts during a 10-minute continuous control session").
- ☐ A reference system still on the old OS (the paired workstation on the other bench in the reference case), to compare behaviour and settings.

- [ ] The original application installer, licence and any dongle; the vendor's OS support statement.
- [ ] Instrument network details recorded from the old PC: IP address, mask, the controller's address, and the UDP ports the application uses (from vendor documentation or a capture).
- [ ] Windows 11 Enterprise image approved for lab use; a second NIC on the workstation.
- [ ] A test window agreed with the lab; the instrument and fixture available.

2.3 Risks and how this SOP controls them

Risk	Control
Losing the working Windows 7 configuration	Document and photograph it first; keep the reference system untouched until OQ passes.
Instrument dropouts during a test	Firewall, power and link settings configured in Chapter 3; 10+ minute OQ run.
Unvalidated change reaching production	Test machine first; release only after QA approval of the summary report.
Legacy VM becomes a security hole	VM has no corporate network, no internet, no shared clipboard or drives; exit date set.

CHAPTER 3

Common Foundation · Workstation and Instrument Network

Every method starts from the same clean, documented Windows 11 workstation with a correctly configured instrument NIC. The steps below follow the sequence used in the reference engagement.

1. **Request decommission and reimage** through the ITSM portal, with the justification "legacy instrument software qualification on Windows 11" and the target image (Windows 11 Enterprise).
2. **Remove the PC from the domain** (or have the endpoint team do it) so the reimaged build starts with a fresh computer object and current policy.
3. **Reimage** and confirm a clean first boot: no provisioning errors, device fully enrolled in endpoint management, current cumulative update installed.
4. **Identify the two NICs.** Rename them so scripts and evidence are unambiguous: `Corporate` and `Instrument`.
5. **Configure the instrument NIC** with a static address, no gateway and no DNS (script below, or **Settings › Network & internet › Ethernet › IP assignment › Edit** as shown after the script).
6. **Set the network profile, firewall rule and power settings** so UDP traffic is neither blocked nor interrupted.

3.1 Base template · prepare the workstation with PowerShell

INITIALIZE-LABINSTRUMENTPC.PS1 · BASE TEMPLATE POWERSHELL

```powershell
<#
.SYNOPSIS  Prepare a Windows 11 lab PC for a legacy instrument application: instrument NIC, firewall,
power,
           optional Windows 7 compatibility layer, and an IQ evidence bundle.
.NOTES     Run elevated on the lab workstation. Access: local Administrators.
#>
[CmdletBinding(SupportsShouldProcess)]
param(
  [Parameter(Mandatory)] [string] $InstrumentAdapter,                # current name, e.g. 'Ethernet 2'
  [string]   $InstrumentIp      = '192.168.10.150',
  [int]      $PrefixLength      = 24,
  [string]   $ControllerIp      = '192.168.10.100',
  [string]   $AppExe            = 'C:\Program Files (x86)\AMTI\NetControl\NetControl.exe',   #
placeholder path
  [int[]]    $UdpPorts          = @(),                               # from vendor docs; empty = allow
by program
  [switch]   $ApplyWin7Layer,
  [string]   $ChangeRef         = 'CHG0000000',
```

```
  [string]   $EvidenceFolder    = 'C:\Validation\Evidence'
)
$ErrorActionPreference = 'Stop'
$alias = 'Instrument'
$ev = Join-Path $EvidenceFolder "$ChangeRef-$(Get-Date -Format yyyyMMdd_HHmmss)"
New-Item -ItemType Directory -Path $ev -Force | Out-Null

# ---- Stage 1: instrument NIC: static IP, no gateway, no DNS registration --------------
if ($InstrumentAdapter -ne $alias -and $PSCmdlet.ShouldProcess($InstrumentAdapter, "Rename to $alias")) {
  Rename-NetAdapter -Name $InstrumentAdapter -NewName $alias
}
if ($PSCmdlet.ShouldProcess($alias, "Set static $InstrumentIp/$PrefixLength")) {
  Set-NetIPInterface -InterfaceAlias $alias -Dhcp Disabled
  Get-NetIPAddress -InterfaceAlias $alias -AddressFamily IPv4 -ErrorAction SilentlyContinue |
    Remove-NetIPAddress -Confirm:$false
  New-NetIPAddress -InterfaceAlias $alias -IPAddress $InstrumentIp -PrefixLength $PrefixLength | Out-Null
# no -DefaultGateway
  Set-DnsClientServerAddress -InterfaceAlias $alias -ResetServerAddresses
  Set-DnsClient -InterfaceAlias $alias -RegisterThisConnectionsAddress $false
}

# ---- Stage 2: network profile, firewall, power ----------------------------------------
if ($PSCmdlet.ShouldProcess($alias, 'Profile Private, UDP allow rule, disable NIC power saving')) {
  Set-NetConnectionProfile -InterfaceAlias $alias -NetworkCategory Private
  $rule = @{ DisplayName = "Instrument UDP ($ChangeRef)"; Direction = 'Inbound'; Protocol = 'UDP'
             InterfaceAlias = $alias; RemoteAddress = $ControllerIp; Action = 'Allow'; Profile = 'Any' }
  if ($UdpPorts) { $rule.LocalPort = $UdpPorts } else { $rule.Program = $AppExe }
  New-NetFirewallRule @rule | Out-Null
  Disable-NetAdapterPowerManagement -Name $alias
  powercfg /change standby-timeout-ac 0          # no sleep during long tests
}

# ---- Stage 3 (optional): Windows 7 compatibility layer for all users ------------------
if ($ApplyWin7Layer -and $PSCmdlet.ShouldProcess($AppExe, 'Set ~ WIN7RTM layer (HKLM)')) {
  $k = 'HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\AppCompatFlags\Layers'
  New-Item -Path $k -Force | Out-Null
  New-ItemProperty -Path $k -Name $AppExe -Value '~ WIN7RTM' -PropertyType String -Force | Out-Null
}

# ---- Stage 4: IQ evidence bundle ------------------------------------------------------
Get-ComputerInfo -Property OsName, OsVersion, OsBuildNumber, WindowsVersion, CsName |
  ConvertTo-Json | Out-File "$ev\os.json"
Get-NetIPConfiguration -InterfaceAlias $alias | Out-String | Out-File "$ev\instrument-nic.txt"
Get-NetAdapter | Select-Object Name, InterfaceDescription, MacAddress, LinkSpeed, Status |
  Export-Csv "$ev\adapters.csv" -NoTypeInformation
Get-CimInstance Win32_PnPSignedDriver | Where-Object DeviceClass -in 'NET', 'USB', 'PORTS' |
  Select-Object DeviceName, DriverVersion, DriverDate | Export-Csv "$ev\drivers.csv" -NoTypeInformation
if (Test-Path $AppExe) { (Get-Item $AppExe).VersionInfo | Select-Object FileVersion, ProductVersion |
  ConvertTo-Json | Out-File "$ev\app-version.json" }
Get-NetFirewallRule -DisplayName "Instrument UDP ($ChangeRef)" -ErrorAction SilentlyContinue |
  Out-String | Out-File "$ev\firewall.txt"
Test-Connection $ControllerIp -Count 4 -ErrorAction SilentlyContinue | Out-File "$ev\ping-controller.txt"
Get-FileHash "$ev\*" | Export-Csv "$ev\_hashes.csv" -NoTypeInformation    # integrity of the evidence set
Write-Host "Evidence written to $ev" -ForegroundColor Green
```

EXAMPLE · RUNNING THE TEMPLATE

INVOCATION (ELEVATED) POWERSHELL

```
.\Initialize-LabInstrumentPC.ps1 -InstrumentAdapter 'Ethernet 2' -InstrumentIp 192.168.10.150 `
   -ControllerIp 192.168.10.100 -ChangeRef CHG0012345 -WhatIf
.\Initialize-LabInstrumentPC.ps1 -InstrumentAdapter 'Ethernet 2' -ChangeRef CHG0012345 -
ApplyWin7Layer
```

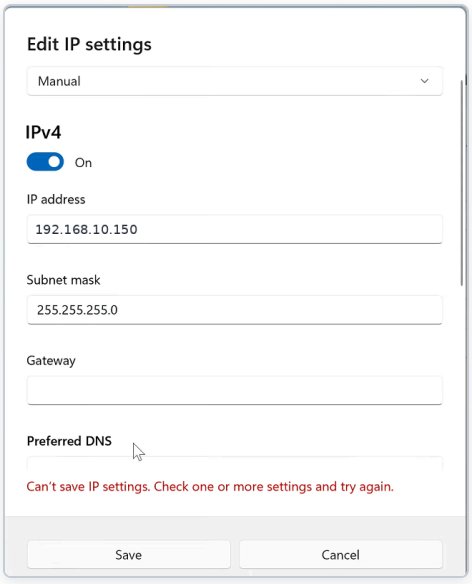


Figure 6. Manual IPv4 on the instrument NIC: address and mask only, gateway and DNS blank. Some Windows 11 builds refuse to save this form ("Can't save IP settings"); use `ncpa.cpl` or PowerShell instead.

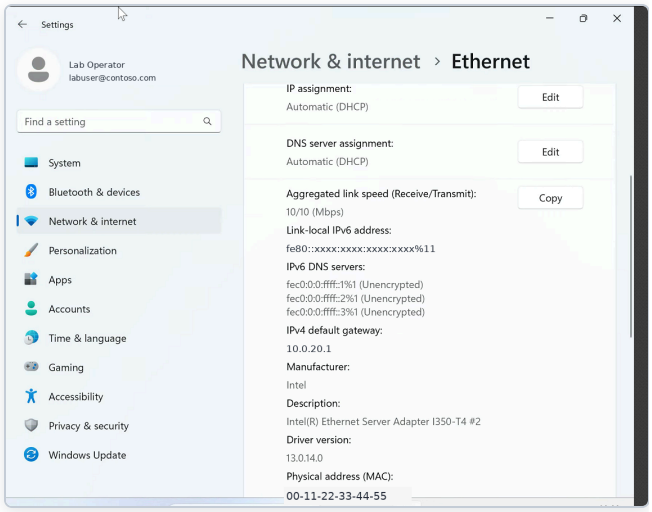


Figure 7. Corporate NIC details in Settings (user, gateway and MAC replaced). Note the 10/10 Mbps link speed reported on this multi-port server adapter: check speed and duplex on every lab link.

NOTE · SETTINGS THAT ARE DRIVER- OR POLICY-SPECIFIC

If the controller is a 10 Mbps device, fix speed and duplex rather than relying on auto-negotiation: list options with `Get-NetAdapterAdvancedProperty -Name Instrument` and set the matching value (display names such as *Speed & Duplex* vary by driver). If Group Policy forces unidentified networks to Public or ignores local firewall rules, ask the endpoint team for a policy exception or a centrally deployed rule.

CHAPTER 4

Method A · Direct Install with Compatibility Mode

Always try the simplest thing first. In the reference engagement, a direct install on Windows 11, with compatibility mode only where needed, was all it took once the instrument network was configured correctly.

1. Copy the original installer locally (not from a network share; some legacy installers fail on UNC paths).
2. Run the installer as administrator. If it refuses ("unsupported operating system") or fails part-way, right-click it › **Properties** › **Compatibility**, tick **Run this program in compatibility mode for** › *Windows 7*, and run it again. Accept the Program Compatibility Assistant's offer if it appears.
3. Complete the standard install: same options, install path and components as on the reference system. Record the installer's file name, version and hash in the IQ evidence.
4. Launch the application. If it fails to start or misbehaves, apply compatibility mode to the *program* executable as well. For a single test, the Compatibility tab is fine; for production, use the all-users setting (Method B) so every operator gets the same behaviour.
5. Restore application configuration from the reference system: station definitions, calibration files and test programs, through the lab's controlled document process.
6. Connect to the instrument and run the OQ checks (Chapter 7).

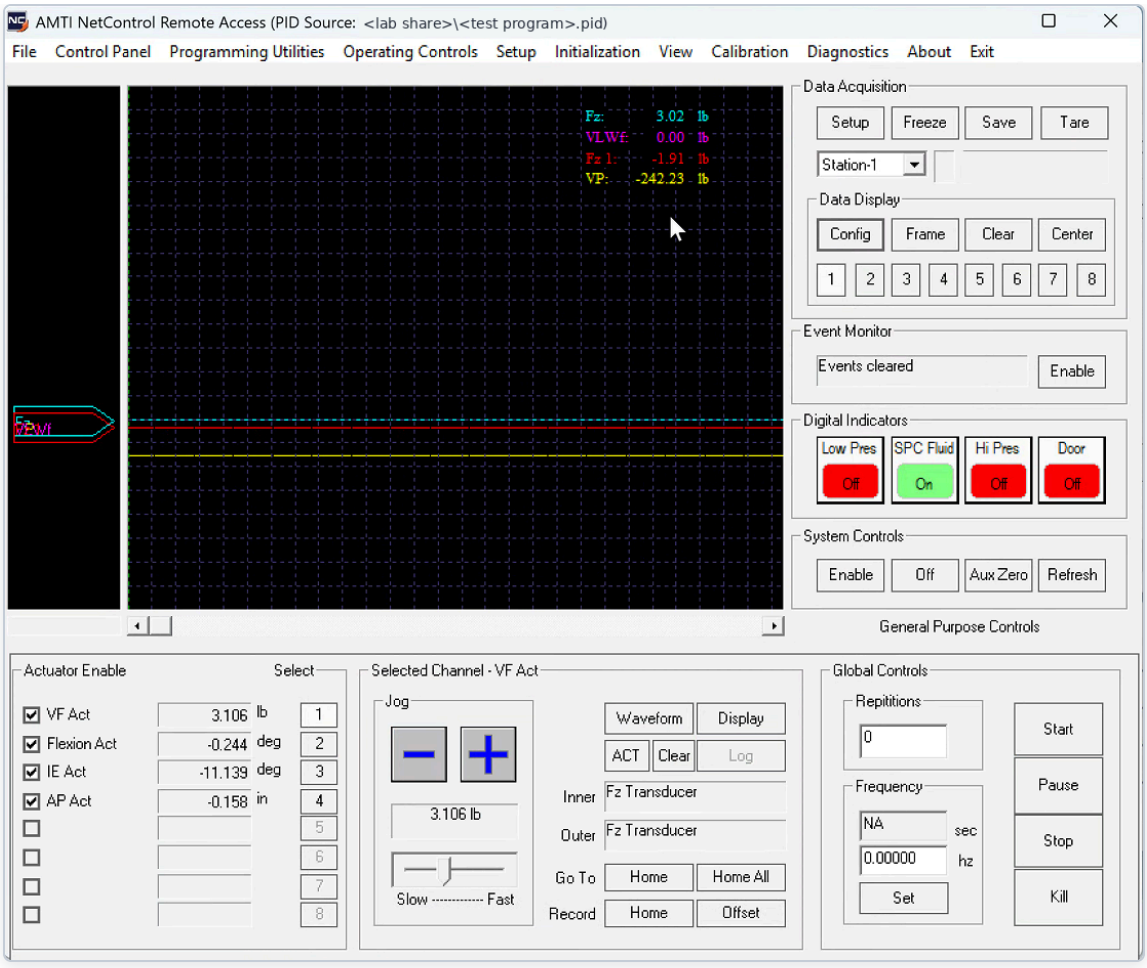


Figure 8. AMTI NetControl running on Windows 11 and communicating with the test frame: live channel values, digital indicators and actuator readings updating (lab file path removed from the title bar).

EXAMPLE · REFERENCE OUTCOME

NetControl installed and ran on Windows 11; the controller was detected and the connection stayed up through extended sessions with no dropouts; no virtual-machine fallback was needed. Earlier Windows 10 attempts had suffered driver instability and UDP sessions that would not persist; on a clean Windows 11 build with the instrument network configured deliberately, the link held. That is why Chapter 3 matters as much as the install.

CHAPTER 5

Method B · Compatibility Shims at Scale

5.1 Option 1 · Compatibility layers in the registry

The Compatibility tab writes a value under `AppCompatFlags\Layers`, where the value name is the full path to the executable and the data is a list of layers prefixed with `~`. Writing the same value under `HKLM` applies it to every user of the PC, and a script makes it repeatable across a fleet.

SET-APPCOMPATLAYER.PS1 POWERSHELL

```powershell
param([string]$Exe = 'C:\Program Files (x86)\AMTI\NetControl\NetControl.exe',
      [string]$Layers = '~ WIN7RTM')                # add RUNASADMIN only if the vendor requires it
$key = 'HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\AppCompatFlags\Layers'
New-Item -Path $key -Force | Out-Null
New-ItemProperty -Path $key -Name $Exe -Value $Layers -PropertyType String -Force
Get-ItemProperty -Path $key | Select-Object -Property $Exe        # verify

# Remove (rollback):
# Remove-ItemProperty -Path $key -Name $Exe
```

WATCH OUT · AVOID "RUN AS ADMINISTRATOR" AS A FIX

Granting elevation makes old software work by giving it the power to change anything on the PC. Prefer shims that redirect file and registry writes (below), or folder permissions scoped to the application's data directory. Use `RUNASADMIN` only when the vendor states it is required, and record it in the risk assessment.

5.2 Option 2 · A custom shim database with Compatibility Administrator

When a single layer is not enough, or you want only the specific fixes an application needs, build a shim database. Compatibility Administrator is installed with the Windows Assessment and Deployment Kit (ADK) under *Application Compatibility Tools*; use the 32-bit edition for 32-bit applications.

1. Open Compatibility Administrator on a test PC. Under **Custom Databases**, select **New Database**, then **Fix › Create Application Fix...**.
2. Enter the program name and vendor, and browse to the executable. On the next page choose a compatibility mode (e.g. *Windows 7*), then select individual fixes, for example `CorrectFilePaths` (redirects writes to protected folders) or `VirtualRegistry`, and use **Test Run...** to try them.
3. Accept the matching information (file attributes that identify the exact executable version), save the database as `NetControl-Win11.sdb`, and store it in controlled

documentation.

4. Install on each workstation, silently, then verify in **Settings › Apps** or the registry:

DEPLOY AND REMOVE THE SHIM DATABASE (ELEVATED) CMD

```
sdbinst.exe -q "C:\Deploy\NetControl-Win11.sdb"
:: rollback
sdbinst.exe -u "C:\Deploy\NetControl-Win11.sdb"
```

5. Package the `sdbinst` command in the endpoint management tool (Intune Win32 app, Configuration Manager) so validated machines receive exactly the same fix.

TIP · TREAT THE .SDB AS CONFIGURATION

A shim database changes how a validated application behaves. Version it, reference it in the IQ, and re-test when the application version changes, because the database's matching information may no longer apply to the new executable.

CHAPTER 6

Methods C and D · Hyper-V Fallback and Vendor Upgrade

6.1 Method C · An isolated Windows 7 VM in Hyper-V

Use this only when Methods A and B fail, and treat it as a time-boxed bridge. The physical instrument NIC is bound to an **external virtual switch** that the host itself does not use (`-AllowManagementOS $false`), so the VM, and only the VM, talks to the controller.

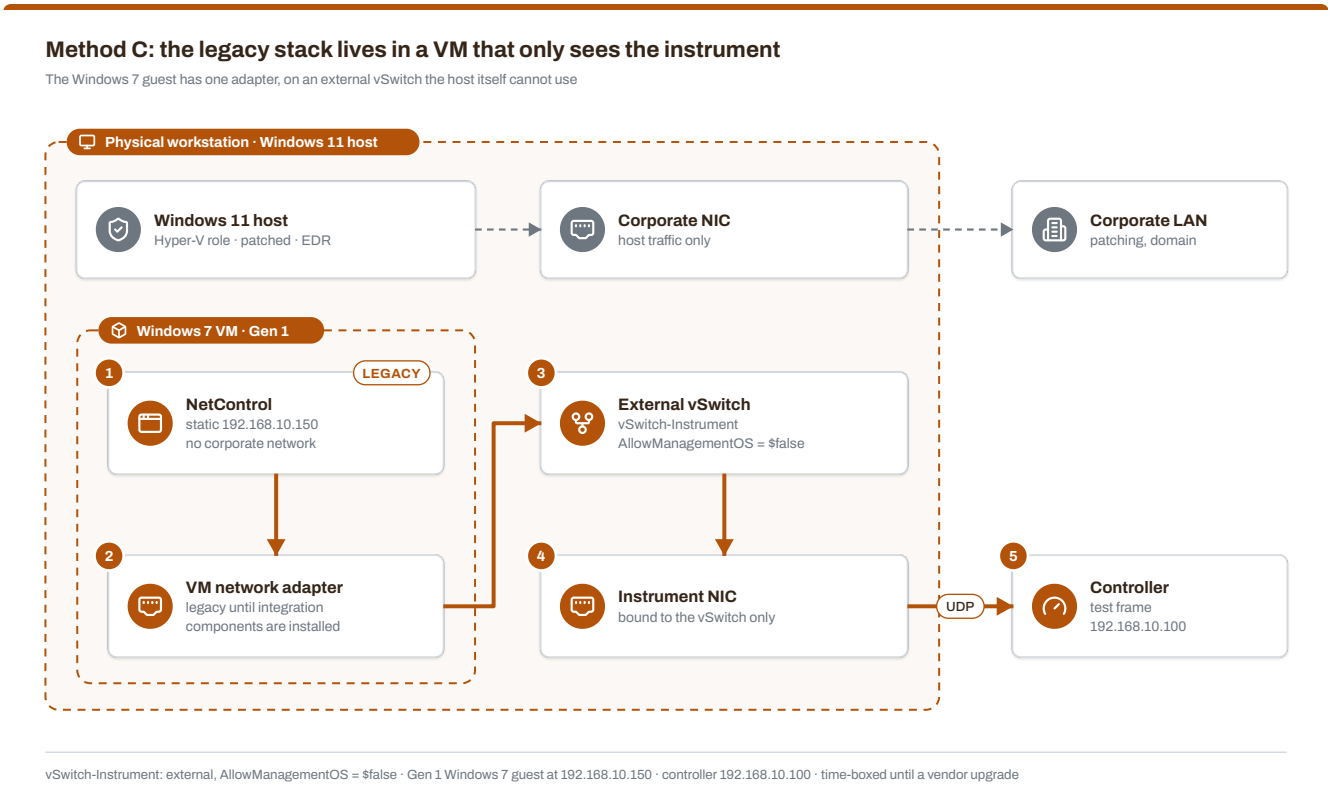


Figure 9. The VM fallback is acceptable only because the unsupported guest can see the instrument and nothing else.

NEW-LEGACYINSTRUMENTVM.PS1 (ELEVATED ON THE WINDOWS 11 HOST) POWERSHELL

```powershell
Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Hyper-V -All   # restart when prompted

$vm = 'VM-W7-INSTR01'; $switch = 'vSwitch-Instrument'
New-VMSwitch -Name $switch -NetAdapterName 'Instrument' -AllowManagementOS $false
New-VM -Name $vm -Generation 1 -MemoryStartupBytes 4GB -SwitchName $switch `
       -NewVHDPath "D:\Hyper-V\$vm.vhdx" -NewVHDSizeBytes 80GB          # Windows 7 requires Generation 1
Set-VMProcessor     -VMName $vm -Count 2
Set-VMMemory        -VMName $vm -DynamicMemoryEnabled $false              # predictable timing
Set-VMDvdDrive      -VMName $vm -Path 'D:\ISO\Windows7-SP1-x64.iso'       # licensed media
Set-VM -Name $vm -AutomaticCheckpointsEnabled $false -AutomaticStartAction Start
# Before integration components are installed in the guest, use a legacy (emulated) adapter:
# Get-VMNetworkAdapter -VMName $vm | Remove-VMNetworkAdapter
# Add-VMNetworkAdapter -VMName $vm -SwitchName $switch -IsLegacy $true
```

```
Start-VM -Name $vm
# After install, inside the guest: static 192.168.10.150/24, no gateway; install the application.
Checkpoint-VM -Name $vm -SnapshotName "Validated baseline $(Get-Date -Format yyyy-MM-dd)"
```

- [ ] No corporate or internet network on the VM; Enhanced Session clipboard and drive redirection disabled for operators.
- [ ] Windows 7 SP1 guest with the Hyper-V integration components update installed (Windows 11 no longer ships the integration-services ISO).
- [ ] USB: Hyper-V on Windows 11 has no general USB pass-through. Instruments that need a USB dongle or USB device cannot use this method without a validated USB-over-IP product.
- [ ] Timing: the virtual switch adds a small latency; the OQ run must prove the controller link is stable.
- [ ] Backup: `Export-VM` the validated VM to controlled storage; checkpoints are not backups.
- [ ] Licensing for the Windows 7 guest is confirmed; an exit date for the VM is in the change record.

6.2 Method D · The vendor upgrade path

The durable answer is a version of the instrument software that the vendor supports on Windows 11. Engage the vendor early, in parallel with Methods A to C:

1. Ask for the official OS support matrix for the current and latest software versions, and whether the controller firmware must also change.
2. Ask whether the vendor has validation support material (IQ/OQ protocols, release notes listing changes that affect data).
3. Assess the impact: test programs and data-file formats, retraining, cost and downtime. Feed this into the risk assessment.
4. Run the upgrade as its own change, using the Common Foundation (Chapter 3) and the OQ in Chapter 7.
5. Decommission any interim VM or shim once the supported version is released.

Method	Security posture	Validation effort	Longevity
A Direct install	Fully patched Windows 11	IQ/OQ of new platform	Until the app breaks on a future update
B Shims	Fully patched; shim reviewed	As A, plus the shim as configuration	Re-test per app/OS update
C Hyper-V VM	Unpatched guest, isolated	IQ/OQ of host and VM	Bridge only; set an exit date
D Vendor upgrade	Supported end to end	Full change incl. data formats	Long term

CHAPTER 7

Verification, Rollback and Troubleshooting

7.1 Operational checks (OQ)

1. Start the application and confirm it detects the controller and shows live values (as in the screenshot in Chapter 4).
2. Run a continuous control session for at least the agreed duration (10+ minutes in the reference engagement) while the monitoring script below records connectivity. Exercise the functions used in real tests: enable actuators, jog, start/stop a waveform, save data.
3. Confirm zero disconnects, timeouts or error events in the application, and zero lost pings in the log.
4. Compare saved data with the same test run on the reference system, if the lab's protocol requires equivalence.

WATCH-INSTRUMENTLINK.PS1 · RUN DURING THE OQ SESSION — POWERSHELL

```
param([string]$Controller = '192.168.10.100', [int]$Minutes = 15, [string]$Out =
'C:\Validation\Evidence\oq-link.csv')
$proc = Get-Process | Where-Object { $_.Path -like '*NetControl*' } | Select-Object -First 1   # adjust
to the app
Get-NetUDPEndpoint -OwningProcess $proc.Id | Format-Table LocalAddress, LocalPort      # ports the app
listens on
$end = (Get-Date).AddMinutes($Minutes)
$rows = while ((Get-Date) -lt $end) {
  $r = Test-Connection $Controller -Count 1 -ErrorAction SilentlyContinue
  [pscustomobject]@{ Time = Get-Date -Format o; Reply = [bool]$r; Ms = $r.Latency   # PowerShell 7 (5.1:
ResponseTime)
                     Link = (Get-NetAdapter -Name Instrument).Status }
  Start-Sleep -Seconds 1
}
$rows | Export-Csv $Out -NoTypeInformation
"Lost: {0} of {1}" -f ($rows | Where-Object { -not $_.Reply }).Count, $rows.Count
```

To see the UDP traffic itself (useful when the vendor asks "are packets arriving?"), use the built-in packet monitor and open the result in Wireshark:

PACKET CAPTURE WITH PKTMON (ELEVATED) — CMD

```
pktmon filter add Instrument -i 192.168.10.100 -t UDP
pktmon start --capture --pkt-size 0 -f C:\Validation\Evidence\oq.etl
:: ... run the session ...
pktmon stop
pktmon etl2pcap C:\Validation\Evidence\oq.etl -o C:\Validation\Evidence\oq.pcapng
pktmon filter remove
```

7.2 Evidence checklist (IQ and OQ)

Installation (IQ)

- [ ] OS edition, version and build (`os.json`, `winver` screenshot)
- [ ] Application version and installer hash
- [ ] Device Manager: NIC and controller drivers with versions
- [ ] Instrument NIC configuration and firewall rule
- [ ] Compatibility setting or .sdb version, if used

Operation (OQ)

- [ ] Screenshot of the application communicating with the instrument
- [ ] Link monitor CSV with zero losses over the agreed duration
- [ ] Optional packet capture
- [ ] Executed test steps with pass/fail, signed and dated
- [ ] Deviations raised and closed

Save evidence to the controlled project location, share the summary with the change owner and the remote-access team, submit the package to Quality, and close the related requests once the change is approved.

7.3 Rollback

Situation	Rollback
OQ fails on Windows 11	Keep the reference Windows 7 system in service (it was never changed); raise a deviation; try Method B or C on the test PC.
Compatibility setting causes new problems	`Remove-ItemProperty` the Layers value, or `sdbinst -u` the database.
VM unstable	`Restore-VMCheckpoint` to the validated baseline, or re-import the exported VM.
Production machine fails after release	Follow the lab's contingency procedure (paired bench or reference system), log the event, reopen the change.

7.4 Troubleshooting

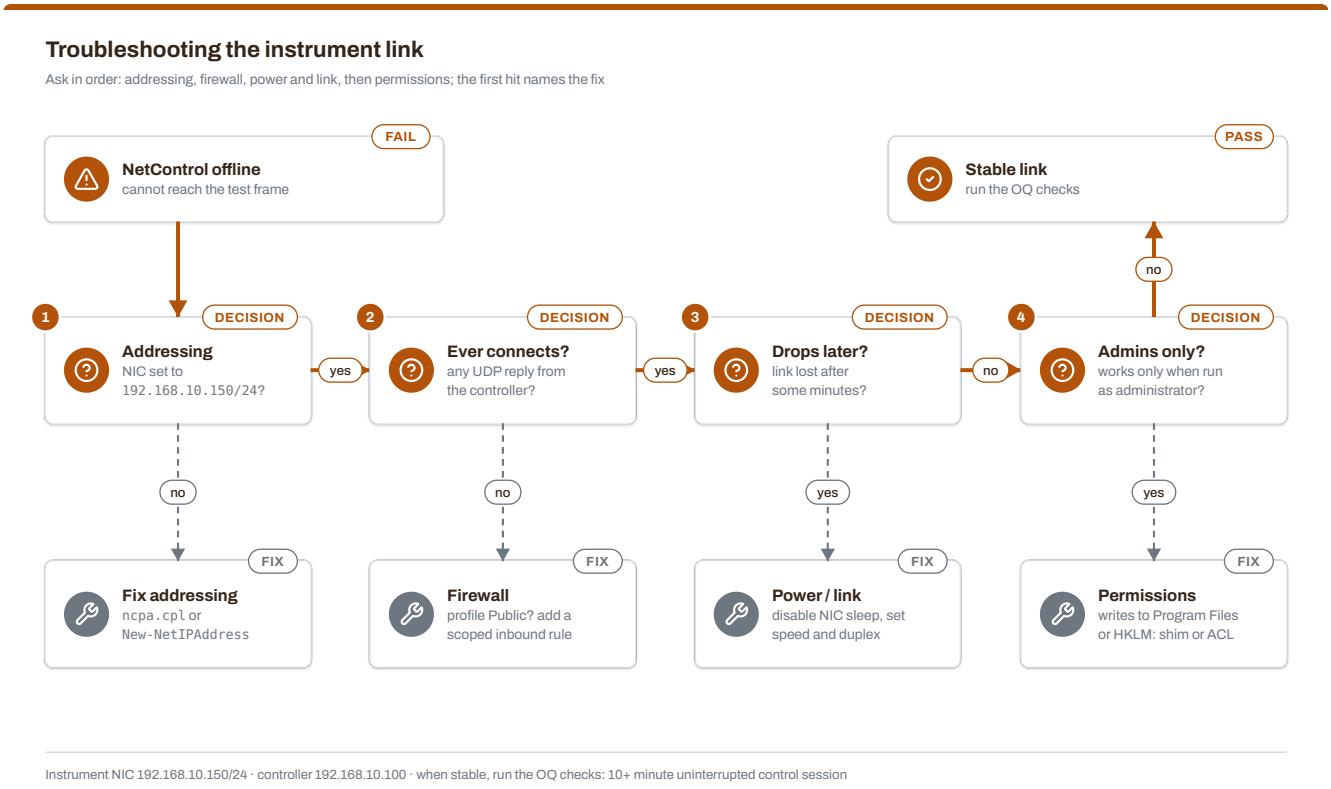


Figure 10. Check the cheapest, most common causes first; each question either clears a layer or points at one specific fix in the table below.

Symptom	Likely cause and fix
Installer exits with "unsupported OS"	Version check. Run the installer in Windows 7 compatibility mode, as administrator.
App starts, controller never connects	Wrong NIC address or subnet, cable on the wrong port, or inbound UDP blocked. Check `Get-NetIPConfiguration -InterfaceAlias Instrument` and the firewall rule; confirm the profile is not Public.
Connects, then drops after minutes	NIC power saving, PC sleep, or speed/duplex mismatch with a 10 Mbps controller. Disable power management, set the link speed explicitly, replace the cable.
Works for admins, not for operators	App writes to Program Files or HKLM. Apply `CorrectFilePaths` / `VirtualRegistry` shims or grant scoped folder permissions.
Corporate network slow or unreachable after setup	A gateway was set on the instrument NIC. Remove it; only the corporate NIC has a default route.
Settings "Can't save IP settings"	Use `ncpa.cpl` (classic adapter properties) or `New-NetIPAddress`.

VM sees no network	Integration components missing in the Windows 7 guest. Use a legacy network adapter, then install the update.

CHAPTER 8

Summary, Glossary and Review

8.1 Chapter summary

- An end-of-life OS in a GxP lab is both a security exposure and a compliance finding; modernize through planned, validated change.
- A dedicated instrument NIC with a static address, no gateway, a Private profile, an explicit UDP rule and power saving off removes the usual causes of "random disconnects".
- A Hyper-V Windows 7 VM is an isolated, time-boxed bridge; the vendor upgrade is the destination.

8.2 Glossary

GxP	Good (Laboratory / Clinical / Manufacturing) Practice regulations.
Shim	Compatibility fix that intercepts API calls between an application and Windows.
WIN7RTM	The compatibility layer behind "Windows 7" compatibility mode.
IQ / OQ / PQ	Installation, operational and performance qualification.

Check your understanding

1. Give two reasons, one security and one regulatory, why a Windows 7 instrument PC must be modernized.
2. What does "Windows 7 compatibility mode" actually change, and what can it not fix?
3. Why does the instrument NIC have no default gateway, and why might Windows block the controller's UDP traffic on a new build?
4. When is the Hyper-V Windows 7 VM appropriate, and what three controls must accompany it?
5. Which evidence belongs in the IQ and which in the OQ for this change?